Riassunti e Appunti di Deep Learning

Appunti di corso



Università di Catania

Università degli Studi di Catania
Dipartimento di Matematica e Informatica
Corso di Laurea Triennale in Informatica (L-31)

Autore
Emanuele Galiano

Anno accademico
Anno Accademico 2026/2027

Versione: 18 ottobre 2026

Prefazione

Queste dispense sono nate come appunti personali per il corso di *Deep Learning* tenuto presso il Dipartimento di Matematica e Informatica dell'Università di Catania dal Prof. **[Nome docente]** nell'anno accademico 2026/2027. L'obiettivo principale di questo materiale è stato fornire una risorsa personale di studio sulla parte teorica del corso, raggruppando concetti chiave, definizioni, teoremi, immagini ed esempi in un unico documento, e integrando gli appunti presi a lezione con le slide fornite dal docente.

Questo file non deve essere visto come un testo ufficiale o completo sull'argomento, in quanto potrebbero esserci errori, omissioni o imprecisioni. Invito pertanto chi legge questo documento a consultare le dispense ufficiali del corso proposte dal docente ed eventuali testi di riferimento consigliati. Inoltre, invito chiunque noti errori o abbia suggerimenti a contattarmi via email:

- **Email personale**: galianoo.emanuele@gmail.com,
- **Email universitaria**: emanuele.galiano@studium.unict.it;

oppure se ancora presente online, aprire una **issue** o una **pull request** nel repository GitHub associato a queste dispense:

`https://github.com/emanuelegaliano/Deep-Learning`

In particolare, all'interno del repository GitHub sono presenti il codice open-source dei sorgenti LaTeX utilizzati per generare queste dispense, oltre a tutti i file di supporto come immagini e codici per la generazione di alcuni grafici presenti nel testo.

Licenza

Questo documento, intitolato *Riassunti e Appunti di Deep Learning – Appunti di corso*, è distribuito sotto licenza **Creative Commons Attribution–ShareAlike 4.0 International (CC BY-SA 4.0)**.

La presente licenza è stata scelta con l'obiettivo di favorire la diffusione, la condivisione e il riutilizzo del materiale didattico, garantendo al contempo il riconoscimento degli autori originali e la preservazione della natura open delle opere derivate.

Autori: Emanuele Galiano
Affiliazione: Università degli Studi di Catania – Dipartimento di Matematica e Informatica
Anno accademico: Anno Accademico 2026/2027

Diritti concessi

In conformità con i termini della licenza Creative Commons BY-SA 4.0, è consentito a chiunque di:

- copiare e ridistribuire il materiale in qualsiasi mezzo o formato;
- adattare, modificare e trasformare il materiale;
- utilizzare il materiale anche per scopi commerciali.

Tali diritti sono concessi a titolo gratuito e non possono essere revocati, purché siano rispettate le condizioni indicate nella sezione seguente.

Condizioni

L'utilizzo del materiale è subordinato al rispetto delle seguenti condizioni:

- **Attribuzione (BY):** deve essere fornita un'adeguata attribuzione dell'opera, citando **titolo**, **autori** e, ove possibile, la **fonte**. L'attribuzione deve essere effettuata in modo ragionevole e non tale da suggerire che gli autori originali approvino l'uso o le modifiche apportate.

- **Condividi allo stesso modo (SA):** nel caso in cui il materiale venga modificato, trasformato o utilizzato per creare opere derivate, tali opere devono essere distribuite sotto la *stessa licenza* Creative Commons Attribution–ShareAlike 4.0 International.

Non è consentito applicare termini legali o misure tecnologiche che limitino giuridicamente altri utenti dall'esercitare i diritti concessi dalla licenza.

Assenza di garanzia

Il materiale è fornito *"così com'è"*, senza garanzie di alcun tipo, esplicite o implicite. In particolare, gli autori non garantiscono l'accuratezza, la completezza o l'assenza di errori nel contenuto del documento e declinano ogni responsabilità per eventuali danni derivanti dall'uso del materiale.

Testo completo della licenza

Il testo legale completo della licenza Creative Commons Attribution–ShareAlike 4.0 International è disponibile al seguente indirizzo:

`https://creativecommons.org/licenses/by-sa/4.0/`

Copyright © 2026
Emanuele Galiano

Indice

Licenza **v**

Diritti concessi v
Condizioni v
Assenza di garanzia vi
Testo completo della licenza vi

I Core Models and Methods **1**

1 Ripasso: dal Machine Learning alle reti neurali **3**

1.1 Gli ingredienti di un algoritmo di Machine Learning 3
1.2 Regressione logistica 4
1.3 Softmax 5
1.3.1 Il modello 5
1.3.2 Funzione di costo e learning 6
1.3.3 Overparametrizzazione 7
1.3.4 Il softmax come rete neurale 7
1.4 Loss function e Multi-Class SVM 8
1.5 Il ruolo della rappresentazione 10
1.6 Reti neurali 10
1.6.1 Multi-Layer Perceptron 10
1.6.2 Le due parti di una rete neurale 12
1.6.3 Funzioni di attivazione 12
1.6.4 Il teorema di approssimazione universale 13
1.6.5 Classificazione multiclasse e regressione 14
1.7 Training e inferenza 15
1.8 Come si apprendono i parametri? 16

2 Backpropagation **17**

2.1 Il problema dell'apprendimento 17
2.2 Grafi computazionali 19
2.2.1 B-diagram 20
2.2.2 Valutazione in due passi 20
2.2.3 L'algoritmo sul grafo 22
2.2.4 Il comportamento delle porte 24

Bibliografia **25**

Parte I

Core Models and Methods

Capitolo 1

Ripasso: dal Machine Learning alle reti neurali

Il Deep Learning non nasce dal nulla: le reti neurali profonde riutilizzano quasi tutti gli strumenti già visti nel Machine Learning classico, ovvero un modello parametrico, una funzione di loss da minimizzare, la discesa del gradiente e la regolarizzazione. In questo capitolo riprendiamo questi strumenti partendo dalla regressione logistica e dal modello softmax, mostriamo che il softmax può già essere visto come una rete neurale molto semplice e arriviamo, attraverso il problema della rappresentazione dei dati, al Multi-Layer Perceptron. Il capitolo si chiude con la domanda che motiva tutto il capitolo successivo: come si calcolano i gradienti di una rete con uno o più hidden layer?

1.1 Gli ingredienti di un algoritmo di Machine Learning

Qualunque algoritmo di Machine Learning supervisionato che vedremo è costruito a partire dagli stessi elementi:

- **Un modello**: una funzione h_ϑ che, dato un input x, restituisce una predizione.
- **Dei parametri**: il vettore (o la matrice) ϑ che determina il comportamento del modello e che va appreso dai dati.
- **Un set di dati**: il training set $\mathcal{D} = \{(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})\}$, composto da m esempi etichettati.
- **Un algoritmo di learning**: la procedura che, minimizzando una funzione di costo $J(\vartheta)$, trova i parametri migliori per il modello.

Ogni input $x^{(i)}$ è un vettore di n feature. Per semplificare la notazione si aggiunge in testa una feature fittizia $x_0^{(i)} = 1$, detta **bias**, in modo che il termine noto del modello possa essere trattato come un parametro qualunque:

$$x^{(i)} = \begin{bmatrix} x_0^{(i)} \\ x_1^{(i)} \\ \vdots \\ x_n^{(i)} \end{bmatrix} \in \mathbb{R}^{n+1}, \qquad x_0^{(i)} = 1$$

dove $x_1^{(i)}, \dots, x_n^{(i)}$ sono le feature vere e proprie dell'i-esimo esempio.

Discesa del gradiente. Come algoritmo di learning useremo quasi sempre la **discesa del gradiente**, ovvero un metodo iterativo che aggiorna i parametri nella direzione opposta al gradiente della funzione di costo:

$$\vartheta_j \leftarrow \vartheta_j - \alpha \frac{\partial J(\vartheta)}{\partial \vartheta_j} \qquad \forall j \text{ simultaneamente}$$

dove $\alpha > 0$ è il **learning rate**, un iperparametro che controlla l'ampiezza del passo. Valori tipici si scelgono su una scala approssimativamente logaritmica, ad esempio $0.1,\ 0.03,\ 0.01,\ 0.003,\ 0.001$: un α troppo grande rischia di far divergere l'algoritmo, mentre uno troppo piccolo rende la convergenza molto lenta.

Regolarizzazione. L'overfitting è uno dei problemi che si possono incontrare durante il training di un modello. Per contrastarlo si aggiunge alla funzione di costo un **termine di regolarizzazione**, che penalizza parametri di valore elevato:

$$J(\vartheta) = \underbrace{-\frac{1}{m}\Big[\cdots\Big]}_{\text{loss sui dati}} + \underbrace{\frac{\lambda}{2m}\sum_{j=1}^{n}\vartheta_j^2}_{\text{regolarizzazione}}$$

dove λ controlla l'intensità della penalizzazione e, tipicamente, il parametro del bias ϑ_0 non viene regolarizzato. Un λ grande porta a modelli più semplici, ma se è troppo grande il modello rischia di andare in underfitting.

1.2 Regressione logistica

La **regressione logistica** è il framework di learning per distinguere tra due classi, ovvero per la classificazione binaria con $y^{(i)} \in \{0, 1\}$. Il modello applica la funzione **sigmoide** a una combinazione lineare delle feature:

$$h_\vartheta(x) = \sigma(\vartheta^T x) = \frac{1}{1 + e^{-\vartheta^T x}}, \qquad \vartheta = [\vartheta_0, \vartheta_1, \dots, \vartheta_n]^T \in \mathbb{R}^{n+1}$$

e restituisce quindi un valore in $(0, 1)$, interpretabile come la probabilità $P(y = 1 \mid x)$.

La funzione di costo è la **cross-entropy loss** (o binary cross-entropy):

$$J(\vartheta) = -\frac{1}{m}\sum_{i=1}^{m}\Big[y^{(i)} \log h_\vartheta(x^{(i)}) + (1 - y^{(i)}) \log\big(1 - h_\vartheta(x^{(i)})\big)\Big]$$

che penalizza fortemente le predizioni sicure ma sbagliate: se $y^{(i)} = 1$ e $h_\vartheta(x^{(i)}) \to 0$ (o viceversa) la loss diverge. I parametri si trovano con la discesa del gradiente e, se necessario, con la regolarizzazione vista sopra.

Per eseguire una classificazione multiclasse con la regressione logistica si può ricorrere all'approccio **one vs all**, addestrando un classificatore binario per ogni classe. Questa strategia presenta però diverse criticità: il numero di classificatori cresce con il numero di classi, possono

comparire regioni di incertezza e le probabilità restituite dai singoli classificatori non sono facilmente confrontabili. Per questo motivo si introduce un modello pensato direttamente per il caso multiclasse.

1.3 Softmax

Il **softmax** è il framework di learning per eseguire la classificazione multiclasse, ed è un'estensione della regressione logistica. Le etichette appartengono ora a K classi, $y^{(i)} \in \{0, 1, \dots, K-1\}$, e vengono rappresentate con il **one-hot encoding**, ovvero come un vettore di K componenti tutte nulle tranne quella corrispondente alla classe corretta, che vale 1.

1.3.1 Il modello

Dato un input x, vogliamo stimare la probabilità che appartenga a ciascuna delle classi:

$$P_\vartheta(y = k \mid x) \qquad \forall k = 0, 1, \dots, K-1.$$

La nostra ipotesi deve quindi stimare un vettore di K probabilità, e per farlo ogni classe k ha il proprio vettore di parametri $\vartheta^{(k)} \in \mathbb{R}^{n+1}$.

Definizione 1.1 (Modello softmax). Il modello softmax restituisce la distribuzione di probabilità sulle K classi:

$$h_\vartheta(x) = \begin{bmatrix} P_\vartheta(y = 0 \mid x) \\ P_\vartheta(y = 1 \mid x) \\ \vdots \\ P_\vartheta(y = K-1 \mid x) \end{bmatrix} = \frac{1}{\sum_{k=0}^{K-1} e^{\vartheta^{(k)T} x}} \begin{bmatrix} e^{\vartheta^{(0)T} x} \\ e^{\vartheta^{(1)T} x} \\ \vdots \\ e^{\vartheta^{(K-1)T} x} \end{bmatrix}$$

ovvero, per la singola classe c:

$$h_\vartheta^{(c)}(x) = P_\vartheta(y = c \mid x) = \frac{e^{\vartheta^{(c)T} x}}{\sum_{k=0}^{K-1} e^{\vartheta^{(k)T} x}}.$$

Il denominatore normalizza la distribuzione, per cui, una volta fissati (appresi) i parametri, la somma delle probabilità restituite è sempre pari a 1:

$$\sum_{k=0}^{K-1} P_\vartheta(y = k \mid x) = 1.$$

I parametri del modello non sono più un vettore ma una matrice, in cui la colonna k-esima

contiene i parametri $\vartheta^{(k)}$ della classe k:

$$\vartheta = \begin{bmatrix} \vartheta_0^{(0)} & \vartheta_0^{(1)} & \cdots & \vartheta_0^{(K-1)} \\ \vartheta_1^{(0)} & \vartheta_1^{(1)} & \cdots & \vartheta_1^{(K-1)} \\ \vdots & \vdots & \ddots & \vdots \\ \vartheta_n^{(0)} & \vartheta_n^{(1)} & \cdots & \vartheta_n^{(K-1)} \end{bmatrix} \in \mathbb{R}^{(n+1)\times K}.$$

1.3.2 Funzione di costo e learning

Anche il softmax usa la cross-entropy loss, che confronta il vettore one-hot dell'etichetta con le probabilità uscite dal modello. Per scriverla definiamo la **funzione indicatrice**:

$$\mathbf{1}\{\text{proposizione}\} = \begin{cases} 1 & \text{se la proposizione è vera} \\ 0 & \text{se la proposizione è falsa} \end{cases}$$

La funzione di costo del softmax è quindi:

$$J(\vartheta) = -\frac{1}{m}\left[\sum_{i=1}^{m}\sum_{c=0}^{K-1} \mathbf{1}\{y^{(i)} = c\} \log h_\vartheta^{(c)}(x^{(i)})\right] + \underbrace{\frac{\lambda}{2m}\sum_{c=0}^{K-1}\sum_{j=1}^{n}\left(\vartheta_j^{(c)}\right)^2}_{\text{regolarizzazione}}$$

dove, per ogni esempio, sopravvive solo il termine della classe corretta: la loss è quindi $-\log$ della probabilità assegnata alla classe giusta.

L'obiettivo del learning è trovare i parametri che minimizzano $J(\vartheta)$, cosa che si fa con la discesa del gradiente (o con la sua versione stocastica). La derivata parziale rispetto al generico parametro vale:

$$\frac{\partial J(\vartheta)}{\partial \vartheta_j^{(c)}} = -\frac{1}{m}\sum_{i=1}^{m}\left[\left(\mathbf{1}\{y^{(i)} = c\} - h_\vartheta^{(c)}(x^{(i)})\right) x_j^{(i)}\right] + \frac{\lambda}{m}\vartheta_j^{(c)}$$

e l'aggiornamento viene ripetuto fino a convergenza:

$$\vartheta_j^{(c)} \leftarrow \vartheta_j^{(c)} - \alpha\frac{\partial J(\vartheta)}{\partial \vartheta_j^{(c)}} \qquad \forall j = 0, \ldots, n,\ \forall c = 0, \ldots, K-1 \text{ simultaneamente.}$$

Osservazione (Gradiente rispetto ai logit). Chiamiamo **logit** le quantità $z_k = \vartheta^{(k)T}x$, ovvero i valori prima della normalizzazione. Per un singolo esempio, il gradiente della cross-entropy rispetto a ciascun logit ha una forma molto semplice:

$$\delta_k = \frac{\partial L}{\partial z_k} = p_k - y_k$$

dove p_k è la probabilità predetta e y_k la componente k-esima del vettore one-hot. Il gradiente rispetto al peso che collega la feature x_j al nodo k è poi semplicemente $\delta_k x_j$, mentre quello rispetto al bias è δ_k. È la stessa espressione della formula precedente, letta nodo per nodo: questa idea di *errore locale* propagato all'indietro tornerà nel capitolo 2.

1.3.3 Overparametrizzazione

Il modello softmax è **overparametrizzato**, ovvero, per ogni ipotesi che descrive i dati, esistono infinite configurazioni dei parametri che producono esattamente la stessa mappa dagli input x alle predizioni. Infatti, sottraendo a ogni vettore $\vartheta^{(k)}$ lo stesso vettore arbitrario $\psi \in \mathbb{R}^{n+1}$:

$$\frac{e^{(\vartheta^{(c)}-\psi)^T x}}{\sum_{k=0}^{K-1} e^{(\vartheta^{(k)}-\psi)^T x}} = \frac{e^{\vartheta^{(c)T}x}\, e^{-\psi^T x}}{e^{-\psi^T x}\sum_{k=0}^{K-1} e^{\vartheta^{(k)T}x}} = \frac{e^{\vartheta^{(c)T}x}}{\sum_{k=0}^{K-1} e^{\vartheta^{(k)T}x}}$$

il fattore comune $e^{-\psi^T x}$ si semplifica e l'ipotesi h_ϑ non cambia.

Relazione con la regressione logistica. Questa proprietà permette di mostrare facilmente che, nel caso particolare $K = 2$, il softmax si riduce alla regressione logistica. Scegliendo $\psi = \vartheta^{(1)}$:

$$h_\vartheta(x) = \begin{bmatrix} \dfrac{e^{(\vartheta^{(0)}-\vartheta^{(1)})^T x}}{e^{(\vartheta^{(0)}-\vartheta^{(1)})^T x} + e^{\mathbf{0}^T x}} \\ \dfrac{e^{\mathbf{0}^T x}}{e^{(\vartheta^{(0)}-\vartheta^{(1)})^T x} + e^{\mathbf{0}^T x}} \end{bmatrix} = \begin{bmatrix} 1 - \dfrac{1}{1 + e^{(\vartheta^{(0)}-\vartheta^{(1)})^T x}} \\ \dfrac{1}{1 + e^{(\vartheta^{(0)}-\vartheta^{(1)})^T x}} \end{bmatrix}$$

dove si è usato $e^{\mathbf{0}^T x} = 1$. La differenza $\vartheta^{(1)} - \vartheta^{(0)}$ può essere sostituita da un unico vettore di parametri ϑ, ottenendo proprio $h^{(1)}_\vartheta(x) = \sigma(\vartheta^T x)$ e $h^{(0)}_\vartheta(x) = 1 - h^{(1)}_\vartheta(x)$. Allo stesso modo, per $K = 2$ e senza regolarizzazione, la funzione di costo del softmax diventa:

$$J(\vartheta) = -\frac{1}{m}\sum_{i=1}^{m}\left[(1 - y^{(i)}) \log\left(1 - h^{(1)}_\vartheta(x^{(i)})\right) + y^{(i)} \log h^{(1)}_\vartheta(x^{(i)})\right]$$

ovvero la cross-entropy della regressione logistica: la regressione logistica è quindi un caso particolare del softmax.

1.3.4 Il softmax come rete neurale

Il softmax segue un modello lineare, tuttavia può essere rappresentato come una rete neurale molto semplice, con un solo layer di pesi tra input e output. I suoi *ingredienti* sono tre:

1. **Modello lineare**: per ogni classe k si calcola $z_k = \vartheta^{(k)T} x$.
2. **Logit**: si applica l'esponenziale, e^{z_k}.
3. **Normalizzazione**: si divide per $\sum_j e^{z_j}$, ottenendo probabilità che sommano a 1.

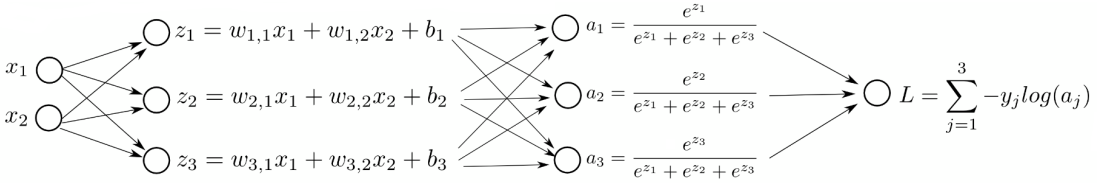


Figura 1.1. Il softmax come rete neurale con due feature e tre classi: dai logit $z_i = w_{i,1}x_1 + w_{i,2}x_2 + b_i$ alle probabilità $a_i = e^{z_i} / \sum_{k=1}^{3} e^{z_k}$, fino alla cross-entropy loss $L = \sum_{j=1}^{3} -y_j \log(a_j)$. Il bias b_i corrisponde al parametro $\vartheta_0^{(i)}$ associato alla feature $x_0 = 1$.

Esempio. Consideriamo tre classi e un input con due feature, $x = [1,\ 0.2,\ 0.1]^T$ (la prima componente è il bias). Siano i parametri, scritti per righe (una riga per classe):

$$\vartheta^T = \begin{bmatrix} -1 & 1 & 0 \\ 1 & 0.5 & 1 \\ 2 & 1 & 1.5 \end{bmatrix}$$

e supponiamo che la classe corretta sia la prima, $y = [1, 0, 0]^T$.

Soluzione. Calcoliamo in ordine i tre ingredienti del softmax:

Classe	Modello lineare z_k	Logit e^{z_k}	Normalizzazione p_k
0	$-1 + 0.2 + 0 = -0.8$	0.45	0.03
1	$1 + 0.1 + 0.1 = 1.2$	3.32	0.23
2	$2 + 0.2 + 0.15 = 2.35$	10.48	0.74
		$\sum = 14.25$	$\sum = 1$

Il modello assegna alla classe corretta una probabilità di appena 0.03, per cui la loss su questo campione è alta: $L = -1 \cdot \log(0.03) \approx 3.46$. La loss complessiva si ottiene facendo la media di questi valori su tutti gli m campioni del training set.

1.4 Loss function e Multi-Class SVM

Ci si potrebbe chiedere se anche una SVM sia una rete neurale. Una SVM lineare, come il softmax, è una **macchina lineare**: calcola un vettore di punteggi $f = Wx + b$, uno per classe. Ciò che cambia tra i due modelli è soltanto la funzione di loss: cambiando la loss di questo framework lineare si ottiene la **Multi-Class SVM**, che usa la **hinge loss**:

$$L = \underbrace{\frac{1}{N} \sum_i \sum_{j \neq y_i} \max(0,\ f_j - f_{y_i} + 1)}_{\text{data loss}} + \underbrace{\lambda \sum_k \sum_l W_{k,l}^2}_{\text{regularization loss}}$$

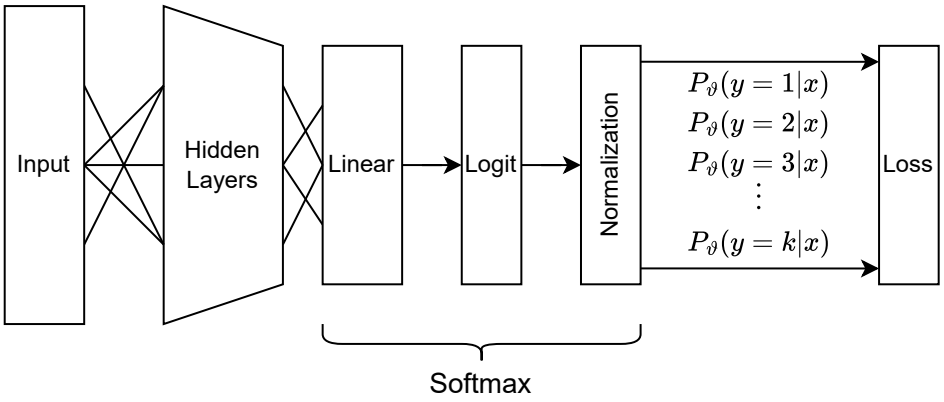


Figura 1.2. Struttura "standard" di una rete neurale per la classificazione: dopo l'input e gli hidden layer, gli ultimi blocchi (modello lineare, logit e normalizzazione) sono esattamente un softmax, che produce le probabilità $P_\vartheta(y = k \mid x)$ su cui viene calcolata la loss.

dove f_{y_i} è il punteggio della classe corretta. In questo senso una SVM lineare può essere vista come la stessa rete a un solo layer del softmax, con una loss diversa; tipicamente, però, il termine "rete neurale" si riserva a modelli che apprendono anche una rappresentazione degli input, come vedremo tra poco.

Esempio. Siano $f = [-2.85,\ 0.86,\ 0.28]^T$ i punteggi prodotti da $Wx + b$ per un esempio la cui classe corretta è $y_i = 2$, ovvero quella con punteggio 0.28.

Soluzione. Con la hinge loss si sommano i contributi delle classi errate:

$$L_{\text{hinge}} = \max(0,\ -2.85 - 0.28 + 1) + \max(0,\ 0.86 - 0.28 + 1) = 0 + 1.58 = 1.58.$$

Con la cross-entropy, invece, si applicano esponenziale e normalizzazione,

$$e^f = [0.058,\ 2.36,\ 1.32]^T \quad \Longrightarrow \quad p = [0.016,\ 0.631,\ 0.353]^T,$$

e si prende il logaritmo della probabilità della classe corretta: $L_{\text{CE}} = -\log(0.353) \approx 1.04$.

Loss per la classificazione binaria. In generale, la loss complessiva è la somma delle loss sui singoli esempi, $L = \sum_{i=1}^{m} \text{loss}\big(f(x^{(i)}), y^{(i)}\big)$. Nel caso binario, con $y^{(i)} \in \{-1, +1\}$, alcuni esempi sono:

- **Zero-one loss**: $\mathbf{1}\{f(x^{(i)}) \neq y^{(i)}\}$, che conta gli errori ma non è né continua né derivabile.
- **Hinge loss**: $\max\big(0,\ 1 - f(x^{(i)})\, y^{(i)}\big)$, usata dalle SVM.
- **Logistic loss**: $\log\big(1 + \exp(-f(x^{(i)})\, y^{(i)})\big)$, usata dalla regressione logistica.

Allo stesso modo, la regressione lineare usa la **least squared loss** $\frac{1}{2}(y - z)^2$, mentre le reti neurali per la classificazione usano la cross-entropy. In tutti i casi la funzione di costo da minimizzare con la discesa del gradiente è $J(\vartheta) = \sum_{i=1}^{m} L\big(h_\vartheta(x^{(i)}), y^{(i)}\big)$, e il modello restituisce un valore continuo nella regressione o una classe nella classificazione.

1.5 Il ruolo della rappresentazione

Un modello lineare come il softmax produce sempre decision boundary lineari: funziona bene quando i dati sono **linearmente separabili**, ma fallisce quando non lo sono, come nel caso di due classi disposte a cerchi concentrici. Una soluzione nota è quella di trasformare le feature con una funzione $\phi(x)$, ad esempio un kernel, che mappa i dati in uno spazio a dimensione più alta in cui diventano separabili da un iperpiano (figura 1.3).

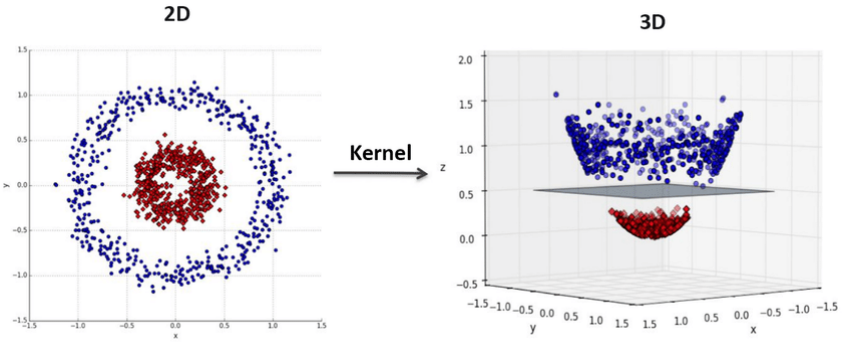


Figura 1.3. Dati non linearmente separabili nello spazio di input bidimensionale (a sinistra): due classi disposte a cerchi concentrici. Applicando una trasformazione ϕ che aggiunge una dimensione (a destra), le due classi diventano separabili da un piano nello spazio delle feature.

Il problema è che scegliere la $\phi(x)$ giusta non è semplice: dipende dai dati e dal task, e una rappresentazione costruita a mano (*hand crafted*) potrebbe non essere quella migliore. La domanda diventa allora: possiamo **apprendere** la funzione $\phi(x)$ direttamente dai dati? La risposta sono le reti neurali e il Deep Learning, che fanno **representation learning**, ovvero imparano in modo supervisionato, a seconda del task, come trasformare l'input in una rappresentazione utile.

1.6 Reti neurali

La prima rete neurale studiata fu il **percettrone** di Rosenblatt, ovvero un singolo neurone che calcola una somma pesata degli input (con la soglia incorporata come peso ϑ_0 del bias $x_0 = 1$) e restituisce un'uscita binaria. Il percettrone è dotato di una procedura di learning automatico dei pesi, ma, come tutti i modelli lineari, risolve solo problemi linearmente separabili: non è in grado, ad esempio, di calcolare lo XOR.

1.6.1 Multi-Layer Perceptron

Il **Multi-Layer Perceptron** (MLP) nasce da due motivazioni:

1. Dobbiamo adattare il modello a dati che vivono in uno spazio non lineare, come nel problema dello XOR.

2. Dato un insieme di dati (ad esempio immagini o documenti di testo), dobbiamo imparare una buona rappresentazione di essi, e non usarne una costruita a mano.

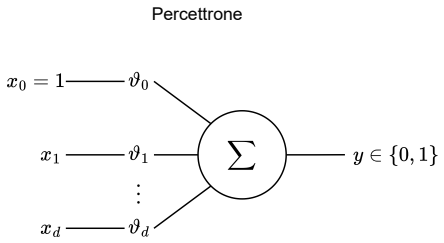


Figura 1.4. Il percettrone di Rosenblatt: le feature $x_1, \dots, x_d$ e il bias $x_0 = 1$ vengono pesati dai parametri $\vartheta_0, \dots, \vartheta_d$ e sommati; l'uscita è binaria, $y \in \{0, 1\}$.

Un MLP è organizzato a strati: un **input layer**, uno o più **hidden layer** e un **output layer**, in cui ogni neurone di un layer è connesso a tutti i neuroni del layer successivo (figura 1.5). Ogni layer, ad eccezione dell'output, contiene anche un'unità di bias che vale sempre 1.

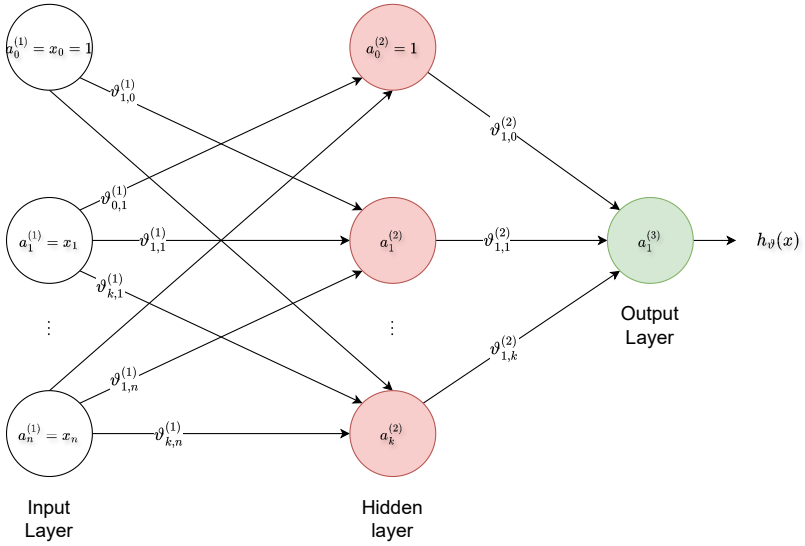


Figura 1.5. Un Multi-Layer Perceptron con un input layer (in bianco, $a_i^{(1)} = x_i$), un hidden layer con k neuroni (in rosso) e un output layer con un solo neurone (in verde) che restituisce $h_\vartheta(x)$. Le unità $a_0^{(1)}$ e $a_0^{(2)}$ sono i bias, sempre pari a 1; i pesi $\vartheta_{ji}^{(l)}$ sono riportati sugli archi.

Notazione. Indichiamo con:

- $a_i^{(l)}$ l'**attivazione** della i-esima unità del layer l; nell'input layer $a_i^{(1)} = x_i$.
- $\vartheta_{ji}^{(l)}$ il parametro che collega l'unità i del layer l (di partenza) all'unità j del layer $l + 1$ (di arrivo).
- $\vartheta^{(l)}$ la matrice dei parametri che mappa il layer l nel layer $l + 1$.

Ogni neurone calcola una combinazione lineare delle attivazioni del layer precedente e vi

applica una **funzione di attivazione** f:

$$z_j^{(l)} = \sum_i \vartheta_{ji}^{(l-1)}\, a_i^{(l-1)} = \vartheta_{j0}^{(l-1)} a_0^{(l-1)} + \vartheta_{j1}^{(l-1)} a_1^{(l-1)} + \dots, \qquad a_j^{(l)} = f\big(z_j^{(l)}\big)$$

o, in forma matriciale, $z^{(l)} = \vartheta^{(l-1)} a^{(l-1)}$ e $a^{(l)} = f(z^{(l)})$, dove f è applicata elemento per elemento. Nel caso della figura 1.5, l'output della rete è quindi:

$$\begin{aligned} h_\vartheta(x) = a_1^{(3)} &= f\big(\vartheta_{10}^{(2)} + \vartheta_{11}^{(2)} a_1^{(2)} + \dots + \vartheta_{1k}^{(2)} a_k^{(2)}\big) \\ &= f\Big(\vartheta_{10}^{(2)} + \vartheta_{11}^{(2)} \underbrace{f\big(\vartheta_{10}^{(1)} + \dots + \vartheta_{1n}^{(1)} a_n^{(1)}\big)}_{a_1^{(2)}} + \dots + \vartheta_{1k}^{(2)} \underbrace{f\big(\vartheta_{k0}^{(1)} + \dots + \vartheta_{kn}^{(1)} a_n^{(1)}\big)}_{a_k^{(2)}}\Big) \end{aligned}$$

Una rete neurale è quindi una **composizione di funzioni derivabili**.

Numero di parametri. Se la rete ha s_l unità nel layer l e s_{l+1} unità nel layer $l+1$, la matrice $\vartheta^{(l)}$ ha dimensione $s_{l+1} \times (s_l + 1)$, dove il $+1$ tiene conto dell'unità di bias. Ad esempio, una rete con 2 feature in input, un hidden layer con 3 neuroni e un output layer con 2 neuroni ha $\vartheta^{(1)} \in \mathbb{R}^{3\times 3}$ e $\vartheta^{(2)} \in \mathbb{R}^{2\times 4}$, per un totale di $9 + 8 = 17$ parametri.

1.6.2 Le due parti di una rete neurale

Una rete neurale per la classificazione è composta da due parti (figura 1.6):

- **Feature learning**: la prima parte della rete impara, in modo supervisionato e a seconda del task, come trasformare l'input x in una rappresentazione utile, nell'esempio $a^{(2)}$.
- **Classificatore**: l'ultima parte è un softmax (o una regressione logistica, nel caso binario) che prende in input le feature apprese.

Si possono aggiungere tutti gli hidden layer necessari per adattare la rete a un problema non lineare, rendendola **profonda** (*deep*). Hidden layer diversi possono avere un numero diverso di neuroni. La funzione di attivazione, invece, è in genere la stessa per tutti i neuroni (si possono usare funzioni diverse, ma si tratta di casi particolari). Al crescere della dimensione dell'input aumenta la probabilità che lo spazio sia non lineare, e può quindi servire aggiungere layer, neuroni per layer, o entrambi.

1.6.3 Funzioni di attivazione

Le funzioni di attivazione **devono essere derivabili**: in questo modo la rete, essendo una composizione di funzioni derivabili, è a sua volta derivabile, e la si può addestrare con la discesa del gradiente. Le più usate sono:

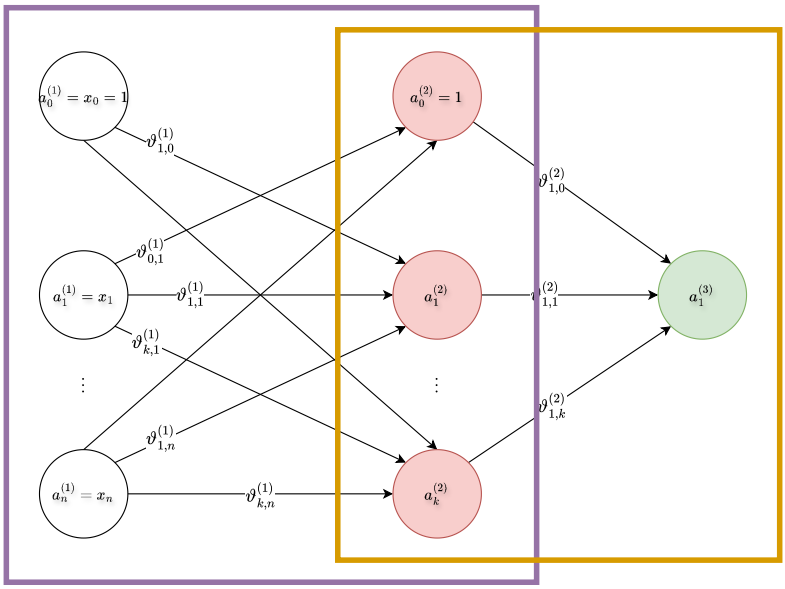


Figura 1.6. Le due parti di una rete neurale: il riquadro viola evidenzia la trasformazione appresa dell'input x nelle feature $a^{(2)}$, mentre il riquadro arancione evidenzia il classificatore (softmax o regressione logistica) che usa le feature apprese.

Nome	Funzione	Derivata
Identità	$f(x) = x$	$f'(x) = 1$
Sigmoide (logistica)	$f(x) = \dfrac{1}{1+e^{-x}}$	$f'(x) = f(x)\big(1 - f(x)\big)$
Tangente iperbolica	$f(x) = \tanh(x) = \dfrac{2}{1+e^{-2x}} - 1$	$f'(x) = 1 - f(x)^2$
ReLU	$f(x) = \max(0, x)$	$f'(x) = \mathbf{1}\{x \geq 0\}$
Softplus	$f(x) = \log(1 + e^x)$	$f'(x) = \dfrac{1}{1+e^{-x}}$

1.6.4 Il teorema di approssimazione universale

Il motivo per cui le reti neurali funzionano così bene su problemi non lineari è formalizzato dal seguente risultato.

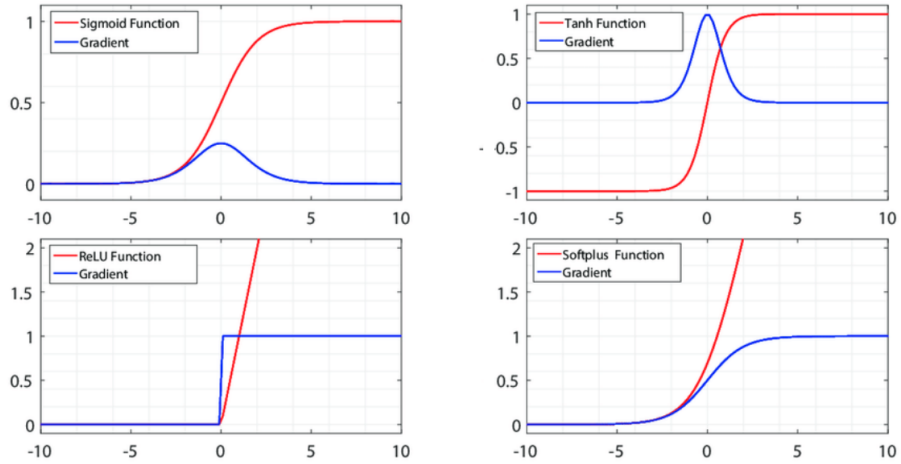


Figura 1.7. Le funzioni di attivazione più usate (in rosso) e le relative derivate (in blu): sigmoide (in alto a sinistra), tangente iperbolica (in alto a destra), ReLU (in basso a sinistra) e softplus (in basso a destra). Si noti come la derivata della softplus sia proprio la sigmoide.

Teorema 1.1 (Approssimazione universale). Una rete neurale feedforward[a] con un solo hidden layer, dotata di un numero finito ma sufficientemente grande di neuroni e di una funzione di attivazione non lineare (ad esempio la sigmoide), è in grado di approssimare qualunque funzione continua $f : K \to \mathbb{R}$ definita su un insieme compatto $K \subset \mathbb{R}^n$ con errore arbitrariamente piccolo, ovvero per ogni $\varepsilon > 0$ esiste una rete che implementa $\hat{f}$ tale che

$$\sup_{x \in K} \left| f(x) - \hat{f}(x) \right| < \varepsilon.$$

[a] Per feedforward si intende una rete in cui ogni layer riceve input esclusivamente dal layer precedente e lo trasmette al successivo, senza cicli.

Il teorema garantisce quindi che un solo hidden layer è sufficiente a *rappresentare* qualunque funzione continua, ma non dice nulla su quanto questo sia praticabile: come osservato in [1], il layer potrebbe dover essere enormemente grande e la rete potrebbe non riuscire ad apprendere e generalizzare correttamente. Il teorema non fornisce infatti né un metodo per trovare i parametri, né un limite al numero di neuroni necessari. In pratica, l'uso di più hidden layer permette spesso di rappresentare le stesse funzioni in modo più compatto: questa è una delle motivazioni delle reti *deep*.

1.6.5 Classificazione multiclasse e regressione

Le reti neurali permettono di addestrare ipotesi complesse e non lineari sullo spazio delle feature, a seconda del task:

- **Classificazione binaria**: basta un solo neurone di output, con $y^{(i)} \in \{0, 1\}$.
- **Classificazione multiclasse**: con $K \geq 3$ classi si usano K unità di output e le etichette sono vettori one-hot, $y^{(i)} \in \{0, 1\}^K$, con un solo 1 nella posizione corrispondente alla classe. Come visto in figura 1.2, l'output layer è un softmax.

- **Regressione**: si eliminano logit e normalizzazione, si usa una regressione lineare nell'ultimo layer (ogni nodo di output è un regressore lineare) e si cambia la funzione di costo, usando la **MSE loss**, che deve tenere conto dell'output di tutti i regressori (figura 1.8).

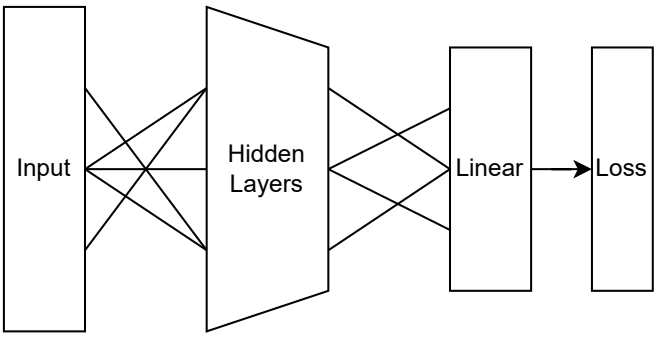


Figura 1.8. Struttura di una rete neurale per la regressione: dopo gli hidden layer l'ultimo blocco è un modello lineare, la cui uscita viene confrontata con il target tramite la MSE loss.

1.7 Training e inferenza

Supponiamo di aver definito un'architettura. Durante la fase di **training** l'obiettivo è imparare i parametri migliori per il modello, e per farlo servono una funzione di loss e un algoritmo che li apprenda. Una volta ottenuti i parametri, questi vengono usati per fare **inferenza** su nuovi pattern.

Funzione di costo. Per la classificazione vogliamo trovare $\hat{\vartheta}$ tale che

$$\hat{\vartheta} = \arg\min_{\vartheta} J(\vartheta), \qquad J(\vartheta) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{c=1}^{K} \mathbf{1}\{y^{(i)} = c\} \log h_{\vartheta}^{(c)}(x^{(i)})$$

ovvero la somma delle cross-entropy di tutti i neuroni di output. Per raggiungere l'obiettivo dobbiamo essere in grado di calcolare in modo efficiente

$$\frac{\partial J(\vartheta)}{\partial \vartheta_{ji}^{(l)}} \qquad \forall j, i, l.$$

Overfitting. Le dimensioni della rete determinano un trade-off. Una rete "piccola" ha pochi parametri, è computazionalmente più economica ma è più soggetta a underfitting. Una rete "grande" ha molti parametri, è più soggetta a overfitting ed è computazionalmente più costosa; in questo caso si usa la regolarizzazione (λ) per contrastare l'overfitting.

Inizializzazione dei parametri. Come inizializzare la matrice dei parametri ϑ? Se si assegnassero gli stessi valori ai pesi che partono dallo stesso nodo, i nodi di arrivo calcolerebbero esattamente le stesse cose e riceverebbero gli stessi gradienti, comportandosi come un unico neurone. Per questo motivo si effettua il **symmetry breaking**: i parametri che dipendono dallo

stesso nodo devono essere inizializzati con valori diversi e piccoli, tipicamente scelti casualmente in un intervallo $\vartheta_{ji}^{(l)} \in [-\varepsilon, \varepsilon]$.

1.8 Come si apprendono i parametri?

Arriviamo così al problema centrale: come si apprendono i parametri di una rete neurale con un numero di hidden layer ≥ 1? Riepilogando quanto visto finora:

- **Un solo neurone**: si usa la discesa del gradiente, evoluzione dell'algoritmo di learning di Rosenblatt.
- **Softmax**: si usa la discesa del gradiente, perché la loss agisce su un solo layer di pesi.
- **Rete con hidden layer**: i valori dei nodi nascosti non sono noti in anticipo, ovvero il training set non fornisce alcuna ground truth per i layer intermedi.

Di conseguenza non possiamo più usare la discesa del gradiente nella forma classica, ma dobbiamo rivedere il modo in cui calcoliamo le derivate rispetto ai parametri. La soluzione si basa sul fatto che le reti neurali sono **grafi computazionali**, su cui si può usare la **chain rule** per spostare i gradienti all'indietro: è l'algoritmo di **backpropagation**, oggetto del prossimo capitolo.

In sintesi, il softmax risolve la classificazione multiclasse ma resta un modello lineare; le reti neurali imparano anche la rappresentazione dei dati e possono approssimare funzioni non lineari arbitrarie, al prezzo di dover calcolare i gradienti rispetto a parametri di layer di cui non conosciamo l'uscita attesa.

Riferimenti

Il materiale visto in questo capitolo comprende:

- Materiale visto a lezione (13 ottobre 2026).
- Slide del corso *Softmax – Generalization of Logistic Regression* e *Neural Network – Multi-Layer Perceptron & Deep Learning*.
- Riassunto dei capitoli 2, 4 e 6 delle dispense di *Machine Learning*, da cui sono tratte anche tutte le figure di questo capitolo: `https://github.com/emanuelegaliano/Machine-Learning`.
- Teorema di approssimazione universale dal libro *Deep Learning* [1], sezione 6.4.1.

Capitolo 2

Backpropagation

Nel capitolo precedente abbiamo visto che una rete neurale con almeno un hidden layer non può essere addestrata con la discesa del gradiente nella forma classica, perché il training set non fornisce alcuna ground truth per i layer intermedi. Eppure la discesa del gradiente ha bisogno, per ogni parametro della rete, della derivata della funzione di costo rispetto a quel parametro. La **backpropagation** è l'algoritmo che permette di calcolare tutte queste derivate in modo efficiente, sfruttando il fatto che una rete neurale è un **grafo computazionale** e che sui grafi computazionali la chain rule si applica in modo del tutto meccanico. In questo capitolo introduciamo i grafi computazionali e il modo in cui si valutano in avanti e all'indietro.

2.1 Il problema dell'apprendimento

Consideriamo una rete come quella in figura 2.1, con 2 feature in input (estese con il bias), un hidden layer con 3 neuroni (più il bias) e un output layer con 2 neuroni. Indichiamo con ϑ l'insieme di tutti i parametri della rete, ovvero le matrici $\vartheta^{(1)} \in \mathbb{R}^{3\times 3}$ e $\vartheta^{(2)} \in \mathbb{R}^{2\times 4}$.

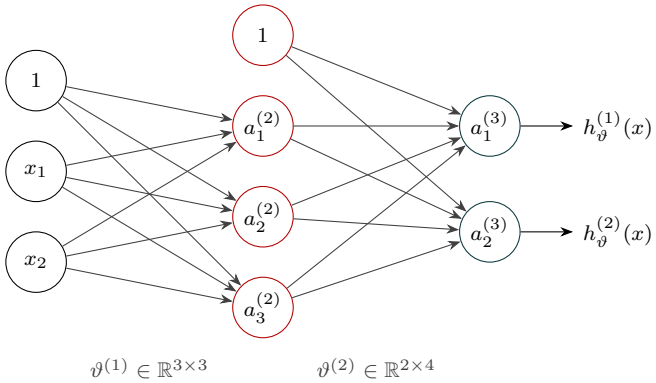


Figura 2.1. Una rete con 2 feature in input, un hidden layer con 3 neuroni (in rosso) e un output layer con 2 neuroni (in verde petrolio); i nodi con valore 1 sono le unità di bias. La matrice $\vartheta^{(1)}$ mappa l'input nell'hidden layer, la matrice $\vartheta^{(2)}$ mappa l'hidden layer nell'output.

Feedforward propagation. Prendendo in esempio la figura 2.1, abbiamo una rete già trainata con cui fare inferenza (quindi con i pesi ϑ fissati). Dato un input $x = [x_1, x_2]^T$, l'output della

rete si calcola propagando in avanti le attivazioni dei neuroni. Le attivazioni del layer di input sono $a_0^{(1)} = x_0 = 1$, $a_1^{(1)} = x_1$ e $a_2^{(1)} = x_2$. Per calcolare le attivazioni dei layer successivi, si moltiplica la matrice dei pesi per il vettore delle attivazioni del layer precedente e si applica la funzione di attivazione f:

$$a_j^{(2)} = f(\sum_{i=0}^{2} \vartheta_{j,i}^{(1)} a_i^{(1)}), \qquad j = 1, 2, 3$$

dove rappresentiamo con $a_i^{(1)}$ le attivazioni del layer di input.

Volendo generalizzare questa forma, dobbiamo considerare una rete con L layer, dove il layer l ha s_l unità. Quindi il layer $l+1$ ha s_{l+1} unità, e la matrice dei pesi $\vartheta^{(l)}$ ha dimensione $s_{l+1} \times (s_l + 1)$, dove il +1 è per il bias. Le attivazioni del layer $l+1$ si calcolano come:

$$a_j^{(l+1)} = f\Big(\sum_{i=0}^{s_l} \vartheta_{j,i}^{(l)} a_i^{(l)} \Big), \qquad j = 1, \dots, s_{l+1},$$

con $a_0^{(l)} = 1$ per ogni l (unità di bias). In forma vettoriale, indicando con $a^{(l)} \in \mathbb{R}^{s_l+1}$ il vettore delle attivazioni del layer l (bias incluso) e con f applicata elemento per elemento:

$$z^{(l+1)} = \vartheta^{(l)} a^{(l)}, \qquad a^{(l+1)} = \begin{bmatrix} 1 \\ f\big(z^{(l+1)}\big) \end{bmatrix}.$$

Il vettore $z^{(l+1)} \in \mathbb{R}^{s_{l+1}}$ contiene le somme pesate prima dell'attivazione. Il prodotto è ben definito perché $\vartheta^{(l)}$ ha $s_l + 1$ colonne, esattamente la lunghezza di $a^{(l)}$.

La feedforward propagation consiste quindi **nel ripetere** questo passo da $l = 1$ a $l = L - 1$, partendo dall'input:

1. $a^{(1)} = [1, x_1, \dots, x_n]^T$, dove $n = s_1$ è il numero di feature.
2. Per $l = 1, \dots, L-1$: calcolare $z^{(l+1)} = \vartheta^{(l)} a^{(l)}$ e $a^{(l+1)} = f\big(z^{(l+1)}\big)$, aggiungendo il bias $a_0^{(l+1)} = 1$ se $l + 1 < L$.
3. L'output della rete è $h_\vartheta(x) = a^{(L)} \in \mathbb{R}^{s_L}$.

Nell'ultimo layer non si aggiunge il bias, perché non c'è un layer successivo a cui trasmetterlo.

Learning. Il problema dell'apprendimento consiste nel trovare l'insieme ottimale di pesi per cui la rete h_ϑ approssima il più possibile una funzione F, rispetto a una funzione di loss. La funzione F non è data esplicitamente, ma solo implicitamente, attraverso gli esempi del training set. I pesi sono numeri reali piccoli, inizializzati casualmente e rispettando il symmetry breaking. Per determinarli bisogna usare la discesa del gradiente per ottenere il minimo della funzione di costo:

$$J(\vartheta) = \frac{1}{2m} \sum_{i=1}^{m} L\big(\vartheta, x^{(i)}, y^{(i)}\big) + \underbrace{\frac{\lambda}{2} \sum_{l=1}^{L-1} \sum_{i=1}^{s_l} \sum_{j=1}^{s_{l+1}} \big(\vartheta_{ji}^{(l)}\big)^2}_{\text{regolarizzazione}}$$

dove L è la loss function, ovvero la cross-entropy per la classificazione o la MSE per la regressione. La discesa del gradiente richiede le derivate di $J(\vartheta)$ rispetto a ogni parametro:

$$\vartheta_{ji}^{(l)} \leftarrow \vartheta_{ji}^{(l)} - \alpha \frac{\partial J(\vartheta)}{\partial \vartheta_{ji}^{(l)}}.$$

Calcolare a mano la derivata per ogni cammino della rete è inefficiente, perché molti cammini condividono gli stessi tratti e gli stessi calcoli verrebbero ripetuti più volte. Serve quindi un algoritmo più furbo, che sfrutti due osservazioni:

1. La rete è una composizione di funzioni derivabili.
2. Vale la proprietà della **chain rule**: se $F(x) = f(g(x))$, allora $F'(x) = g'(x)\, f'\big(g(x)\big)$.

Per trovare un modo efficiente di calcolare il gradiente su una rete si ricorre ai grafi computazionali.

2.2 Grafi computazionali

Definizione 2.1 (Grafo computazionale). Un **grafo computazionale** è un grafo orientato in cui:

- I **nodi** sono unità di calcolo: ogni nodo calcola una singola **funzione primitiva** del proprio input, ad esempio una somma, un prodotto o $1/x$.
- Gli **archi** trasmettono informazione numerica da un nodo all'altro, in avanti; un arco può essere **pesato**, e quando il peso non è indicato vale 1.

Le funzioni primitive devono essere **continue** e **derivabili**.

Una rete neurale feedforward rappresenta una catena di composizioni di funzioni che trasforma un input in un output, ed è quindi un grafo computazionale: i neuroni sono i nodi, i pesi $\vartheta_{ji}^{(l)}$ sono i pesi degli archi e le funzioni di attivazione sono le primitive dei nodi, assunte uguali per tutti i nodi.

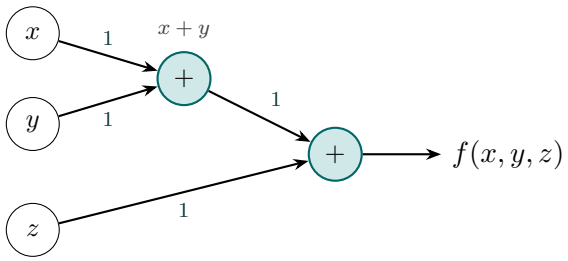


Figura 2.2. Esempio di Grafo Computazionale. Viene rappresentata la funzione $f(x, y, z) = (x + y) + z$: i nodi con lo sfondo verde petrolio calcolano la funzione primitiva somma, gli archi trasmettono i valori da sinistra verso destra e hanno tutti peso 1.

Esempio. Consideriamo la figura 2.2, che rappresenta un grafo computazionale per la funzione

$$f(x, y, z) = (x + y) + z.$$

La somma è una funzione primitiva, per cui la funzione si scompone in due nodi: il primo calcola $x + y$, il secondo somma z al risultato del primo.

2.2.1 B-diagram

Per poter calcolare le derivate, si dà a ogni nodo della rete una struttura composta, ottenendo un **backpropagation diagram** (B-diagram).

Definizione 2.2 (B-diagram). Un **B-diagram** è un grafo orientato in cui ogni nodo è diviso in due metà:

- la parte destra calcola la funzione primitiva del nodo;
- la parte sinistra calcola la derivata della funzione primitiva del nodo.

Gli archi non hanno più una freccia, perché il grafo può essere percorso sia in avanti sia all'indietro (Figura 2.3).

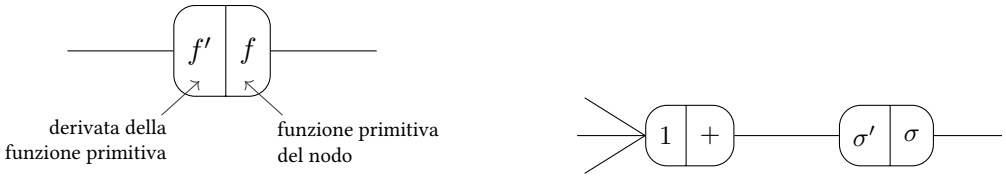


Figura 2.3. A sinistra, un nodo di un B-diagram: la metà destra contiene la funzione primitiva f, la metà sinistra la sua derivata f'. A destra, un neurone con attivazione sigmoide scomposto in due nodi: un nodo di integrazione (somma), la cui derivata rispetto a ogni input è 1, seguito dal nodo di attivazione σ.

Nodi di integrazione e di attivazione. Considerando le funzioni primitive, conviene separare la funzione di integrazione, ovvero la somma degli input di un neurone, dalla funzione di attivazione. Ad esempio, un neurone con attivazione sigmoide, il cui input è una combinazione lineare $z = \sum_i \vartheta_i x_i$, diventa la coppia di nodi a destra in figura 2.3: il nodo somma ha derivata 1 rispetto a ciascun input, mentre il nodo di attivazione calcola

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \sigma'(z) = \sigma(z)\big(1-\sigma(z)\big) = \frac{1}{1+e^{-z}}\left(1 - \frac{1}{1+e^{-z}}\right).$$

Gli archi tra un nodo e l'altro possono essere pesati.

2.2.2 Valutazione in due passi

Per il training, la rete viene valutata in due passi:

1. **Feedforward step**: l'informazione va da sinistra verso destra. In ogni nodo vengono valutate sia le funzioni primitive (lato destro) sia le loro derivate (lato sinistro), e i risultati vengono memorizzati. I risultati del lato destro vengono inoltrati alle unità connesse sulla destra.

2. **Backward step**: serve a valutare gli errori e a propagarli all'indietro per aggiornare i parametri. In questo caso vengono usati i risultati del lato sinistro, ovvero le derivate. L'input dalla destra è sempre la costante 1.

Per capire come si combinano le informazioni nel backward step bisogna considerare tre casi: la composizione di funzioni, la somma di funzioni e gli archi pesati.

Caso 1: composizione di funzioni. Consideriamo due nodi in serie, il primo con primitiva g e il secondo con primitiva f, che calcolano quindi $f(g(x))$ (figura 2.4). Nel feedforward step il primo nodo riceve x, memorizza $g'(x)$ e trasmette $g(x)$; il secondo memorizza $f'(g(x))$ e restituisce $f(g(x))$. Nel backward step si immette 1 da destra: il secondo nodo moltiplica l'input per il valore memorizzato a sinistra e trasmette $1 \cdot f'(g(x))$; il primo fa lo stesso e restituisce

$$1 \cdot f'\big(g(x)\big)\, g'(x)$$

che è esattamente la derivata della piccola rete secondo la chain rule. La backpropagation fornisce quindi un modo per implementare il principio della chain rule.

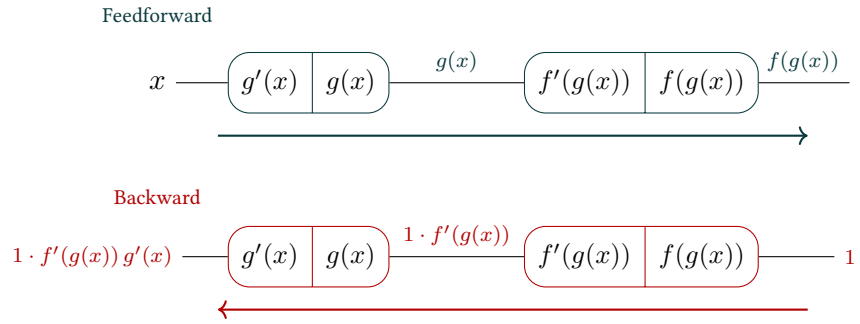


Figura 2.4. Caso 1, composizione di funzioni. In alto il feedforward step (in verde petrolio): ogni nodo memorizza la propria derivata a sinistra e trasmette il valore della primitiva a destra. In basso il backward step (in rosso): partendo dalla costante 1, ogni nodo moltiplica l'input proveniente da destra per la derivata memorizzata, ottenendo all'estremo sinistro la derivata $f'(g(x))\, g'(x)$ della composizione.

In generale, se due nodi in serie hanno memorizzato i valori B' e A' durante il feedforward, il valore che attraversa all'indietro il nodo A è $1 \cdot A'$ e il valore che esce da B è $1 \cdot A' \cdot B'$, ovvero la derivata. Si noti che le derivate $g'(x)$ e $f'(g(x))$ vengono calcolate nel feedforward e usate nel backward, mentre $g(x)$ e $f(g(x))$ vengono calcolate e usate nel solo feedforward.

Caso 2: somma di funzioni. Consideriamo ora un input x che viene inviato a due nodi f_1 e f_2, le cui uscite vengono sommate da un nodo $+$ (figura 2.5). Il nodo somma $F(x_1, x_2) = x_1 + x_2$ ha due derivate parziali, $\partial F/\partial x_1 = \partial F/\partial x_2 = 1$. Nel feedforward la rete calcola $f_1(x) + f_2(x)$. Nel backward step il nodo somma riceve 1 e lo trasmette, moltiplicato per 1, su entrambi i rami; ogni ramo lo moltiplica per la propria derivata, ottenendo $1 \cdot 1 \cdot f_1'(x)$ e $1 \cdot 1 \cdot f_2'(x)$. Andando all'indietro, **i valori che arrivano allo stesso nodo da rami diversi vengono sommati**, per cui all'estremo sinistro si ottiene

$$f_1'(x) + f_2'(x)$$

ovvero proprio la derivata di $f_1(x) + f_2(x)$. Un nodo il cui output viene inviato a più nodi è quindi equivalente a più copie dello stesso nodo, e i gradienti delle copie si sommano.

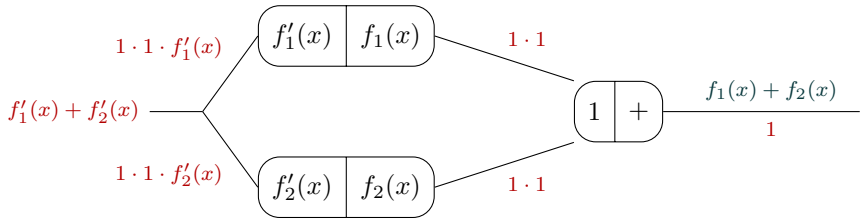


Figura 2.5. Caso 2, somma di funzioni. Il nodo somma restituisce $f_1(x) + f_2(x)$ nel feedforward (in verde petrolio); nel backward (in rosso) la costante 1 viene distribuita su entrambi i rami, ciascuno la moltiplica per la propria derivata e, nel punto in cui i rami si separano, i due contributi vengono sommati.

Caso 3: archi pesati. Infine, consideriamo due nodi A e B collegati da un arco di peso ϑ (figura 2.6). Nel feedforward il nodo A restituisce $A(x)$, l'arco la moltiplica per ϑ e il nodo B riceve $\vartheta A(x)$, restituendo $B(\vartheta A(x))$. Nel backward step il nodo B trasmette $B'(\vartheta A(x))$, l'arco lo moltiplica per ϑ e il nodo A lo moltiplica per la propria derivata, restituendo

$$\vartheta\, B'\big(\vartheta A(x)\big)\, A'(x)$$

che è la derivata di $B(\vartheta A(x))$ rispetto a x. I pesi modulano quindi i valori trasmessi **in entrambe le direzioni**.

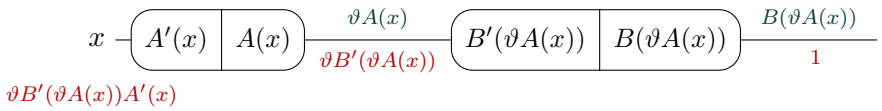


Figura 2.6. Caso 3, archi pesati. Sopra l'arco il valore trasmesso nel feedforward (in verde petrolio), sotto l'arco quello trasmesso nel backward (in rosso): in entrambe le direzioni il valore viene moltiplicato per il peso ϑ.

2.2.3 L'algoritmo sul grafo

Mettendo insieme i tre casi, data una rete F con input x, la derivata $F'(x)$ si calcola come segue:

1. **Forward pass**: l'input viene valutato da sinistra verso destra lungo la rete; i valori delle funzioni primitive e delle loro derivate vengono calcolati e memorizzati in ogni nodo.
2. **Backward pass**: la costante 1 viene imposta come input destro della rete, e la rete viene percorsa da destra verso sinistra. L'informazione che arriva a un nodo viene sommata e il risultato viene moltiplicato per il valore memorizzato nella parte sinistra del nodo e per il peso dell'arco. Il risultato finale è la derivata di F rispetto a x.

Osservazione. Qualunque grafo computazionale, ovvero qualunque composizione di funzioni, può essere valutato con la backpropagation per calcolarne le derivate. In questo senso la backpropagation non è un algoritmo specifico per le reti neurali, ma un modo generale di applicare la chain rule.

Esempio. Consideriamo la funzione

$$f(x, y, z) = (x + y) \cdot z$$

con input $x = -2$, $y = 5$, $z = -4$. Vogliamo calcolare $\frac{\partial f}{\partial x}$, $\frac{\partial f}{\partial y}$ e $\frac{\partial f}{\partial z}$.

Soluzione. Scomponiamo la funzione nelle due primitive $q = x + y$ e $f = q \cdot z$, di cui conosciamo le derivate parziali:

$$\frac{\partial q}{\partial x} = 1, \quad \frac{\partial q}{\partial y} = 1, \qquad \frac{\partial f}{\partial q} = z, \quad \frac{\partial f}{\partial z} = q.$$

Per la chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial x} = z \cdot 1, \qquad \frac{\partial f}{\partial y} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial y} = z \cdot 1, \qquad \frac{\partial f}{\partial z} = q.$$

Sul grafo computazionale (figura 2.7) procediamo in due passi:

1. **Forward pass**: il nodo somma calcola $q = -2 + 5 = 3$, il nodo prodotto calcola $f = 3 \cdot (-4) = -12$.
2. **Backward pass**: si immette 1 a destra. Il nodo prodotto trasmette verso q il valore $1 \cdot z = -4$ e verso z il valore $1 \cdot q = 3$. Il nodo somma riceve -4 e lo trasmette, moltiplicato per 1, sia verso x sia verso y.

Si ottiene quindi $\frac{\partial f}{\partial x} = -4$, $\frac{\partial f}{\partial y} = -4$ e $\frac{\partial f}{\partial z} = 3$.

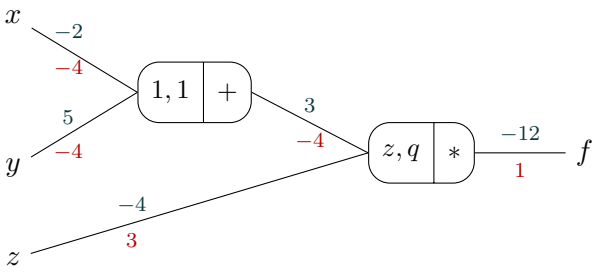


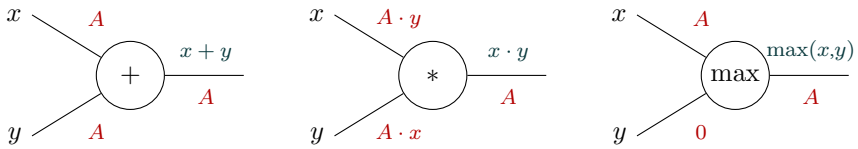
Figura 2.7. Forward e backward pass per $f(x, y, z) = (x + y) \cdot z$ con $x = -2$, $y = 5$, $z = -4$. Sopra gli archi i valori del forward pass (in verde petrolio), sotto gli archi i valori del backward pass (in rosso), che coincidono con le derivate parziali di f rispetto a ogni variabile. Nei nodi, a sinistra, sono riportate le derivate parziali della primitiva rispetto ai due input.

2.2.4 Il comportamento delle porte

L'esempio precedente mostra un comportamento ricorrente dei nodi, o *porte*, più comuni durante il backward pass. Indicando con A il gradiente che arriva da destra (figura 2.8):

- **Add gate**: *distribuisce* il gradiente, trasmettendo A invariato a tutti i propri input.
- **Mul gate**: *scambia* il gradiente, ovvero trasmette a ciascun input A moltiplicato per il valore dell'altro input: $A \cdot y$ verso x e $A \cdot x$ verso y.
- **Max gate**: *instrada* il gradiente, trasmettendo A solo all'input che era il massimo e 0 agli altri, perché $\dfrac{\partial \max(x,y)}{\partial x} = \mathbf{1}\{x \geq y\}$ e $\dfrac{\partial \max(x,y)}{\partial y} = \mathbf{1}\{y \geq x\}$.

Inoltre, come visto nel caso 2, quando l'output di un nodo viene inviato a più rami **i gradienti dei rami si sommano**.



Add gate **Mul gate** **Max gate ($x > y$)**

Figura 2.8. Comportamento delle porte durante il backward pass, con in verde petrolio l'output del forward pass e in rosso i gradienti: la add gate distribuisce il gradiente A, la mul gate lo scambia moltiplicandolo per l'altro input e la max gate lo instrada verso l'input massimo.

Riferimenti

Il materiale visto in questo capitolo comprende:

- Materiale visto a lezione (13 ottobre 2026).
- Slide del corso *Backpropagation (via Computational Graphs).*
- Capitolo 7, *The Backpropagation Algorithm*, del libro *Neural Networks: A Systematic Introduction* [2].

Bibliografia

[1] Ian Goodfellow, Yoshua Bengio e Aaron Courville. *Deep Learning*. MIT Press, 2016. URL: http://www.deeplearningbook.org.

[2] Raúl Rojas. *Neural Networks: A Systematic Introduction*. Berlin, Heidelberg: Springer, 1996. ISBN: 978-3540605058.